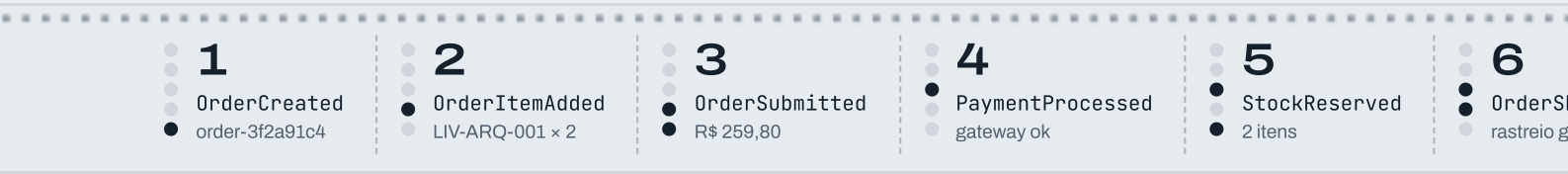


Violet-pixel/
Distributed-Event-Messaging-Engine

Brado

Manual técnico de uso

Como instalar, configurar, operar e integrar o motor de mensageria do Brado: broker TCP, banco de eventos, modelo de leitura, outbox, saga de atendimento e painel de monitoramento.



Versão
1.1.0

Data
Outubro de 2026

Autoria
Copyright (©) 2026 Clara Portes

Sumário

1 **Introdução** 3
O que é o Brado e como o código se divide

2 **Instalação e execução** 4
SQLite, PostgreSQL e Docker

3 **Configuração** 6
Todas as chaves do appsettings.json

4 **Conceitos do broker** 7
Tópicos, offsets, grupos e fila de erros

5 **Protocolo TCP** 9
Layout binário e sequência de quadros

6 **Cliente .NET** 11
Publicar e consumir

7 **API HTTP** 12
Comandos, consultas, eventos e broker

8 **Painel de monitoramento** 14
O que cada área mostra

9 **Event Sourcing e CQRS** 17
Agregado, snapshots e projeções

10 **Saga de atendimento** 18
Fluxo e compensação

11 **Outbox e garantias de entrega** 19
At-least-once sem perder mensagens

12 **Testes e benchmarks** 20
Como rodar e o que cobrem

13 **Operação e solução de problemas** 21
Produção, backup e diagnóstico

14 **Automação de commits** 22
Workflow com cinco execuções por dia

15 **Novidades da 1.1.0** 22
Novo visual do painel e dos manuais

Copyright (©) 2026 Clara Portes

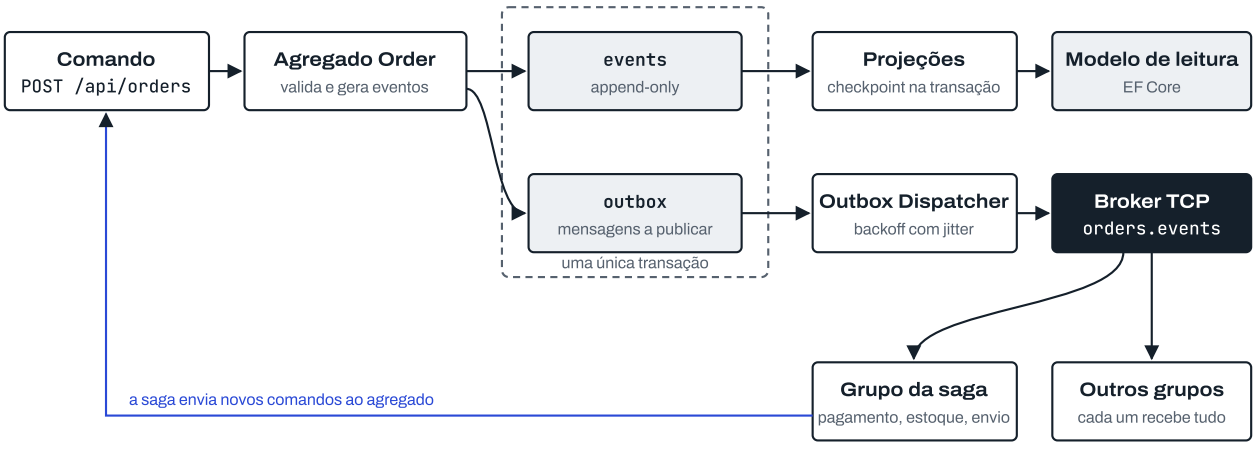
1 Introdução

O Brado junta um broker de mensagens e um banco de eventos num único processo .NET 10. Escrita e leitura ficam separadas, e cada mudança de estado é guardada como um evento que não se altera depois.

O código está dividido em projetos com responsabilidades estreitas. Cada um pode ser usado sozinho: o cliente TCP, por exemplo, só depende do protocolo.

Projeto	Responsabilidade
`Brado.Protocol`	Quadro binário, codec com `Span<T>`, leitor e escritor sobre `Stream`, payloads auxiliares e validação de nomes.
`Brado.Domain`	Agregado `Order`, eventos de domínio, snapshot e o envelope publicado no broker.
`Brado.Broker`	Tópicos, grupos de consumo, servidor TCP (`BrokerServer`), handlers por tipo de quadro e métricas.
`Brado.Client`	Cliente TCP com multiplexação por correlação, assinaturas e ACK automático.
`Brado.EventStore`	Event store, outbox, saga store, inbox de idempotência e journal do broker, com Dapper e SQL.
`Brado.Cqrs`	Comandos, consultas, dispatchers, modelo de leitura em EF Core e motor de projeções.
`Brado.Workers`	Outbox dispatcher, projection worker e saga, todos como `BackgroundService`.
`Brado.Server`	Host ASP.NET Core com a API, os Server-Sent Events, o painel e o simulador de tráfego.

Como as peças se ligam



Um comando passa pelo agregado, que grava eventos e mensagens do outbox na mesma transação. O dispatcher entrega ao broker; as projeções leem o log e atualizam o modelo de leitura.

Copyright (©) 2026 Clara Portes

Quando usar cada parte

Só o broker

Filas e pub/sub entre serviços: publique em tópicos e consuma em grupos com prefetch, ACK e fila de erros.

Event Sourcing

Domínios que precisam de histórico completo e auditável, com o estado reconstruído em qualquer versão.

CQRS

Leituras rápidas em visões desnormalizadas, independentes do modelo de escrita e reconstruíveis por replay.

Outbox e saga

Processos entre serviços sem two-phase commit: cada passo publica eventos com garantia e a saga compensa as falhas.

2 Instalação e execução

Há três jeitos de subir o Brado: SQLite local sem configuração nenhuma, PostgreSQL local ou tudo em contêineres.

Requisitos

- .NET SDK 10. O `global.json` fixa a versão 10.0.100 e aceita versões mais novas (roll forward).
- Docker, para o PostgreSQL e para os testes de integração com Testcontainers.
- Portas livres: 5080 (HTTP e painel) e 7420 (broker TCP).

SQLite (padrão)

```
git clone https://github.com/Violet-pixel/Distributed-Event-Messaging-Engine.git
cd Distributed-Event-Messaging-Engine
dotnet restore Brado.slnx
dotnet run --project src/Brado.Server
```

Os bancos ficam em `data/brado-events.db` (escrita) e `data/brado-read.db` (leitura). O schema é criado na inicialização e pode ser recriado sem risco: o script só cria o que ainda não existe.

PostgreSQL

```
docker compose up -d postgres
dotnet run --project src/Brado.Server --environment Postgres
```

O ambiente `Postgres` carrega `appsettings.Postgres.json`, que aponta o event store para o banco `brado` e o modelo de leitura para `brado_read`. O segundo banco é criado automaticamente.

Copyright (©) 2026 Clara Portes

Docker Compose completo

```
docker compose up --build
```

Sobe o PostgreSQL 17 e o servidor com o tráfego ambiente ligado. O painel fica em `http://localhost:5080`.

Serviço	Endereço	Função
Painel e API	`http://localhost:5080`	Painel, API REST e Server-Sent Events
Broker	`tcp://localhost:7420`	Protocolo binário para publishers e subscribers

> **Modo demonstração.** Abra `http://localhost:5080/?demo` para ver o painel com dados sintéticos. Ele funciona mesmo sem o servidor rodando.

Copyright (©) 2026 Clara Portes

3 Configuração

Todas as opções ficam em `appsettings.json`. Qualquer uma pode ser sobrescrita por variável de ambiente no formato `Secao__Chave`.

Broker

Chave	Padrão	Descrição
`Host / Port`	`0.0.0.0 / 7420`	Endereço de escuta do servidor TCP.
`MaxConnections`	`2048`	Conexões simultâneas. Acima disso o servidor responde `ERROR` e fecha.
`MaxPayloadBytes`	`4 MiB`	Tamanho máximo do payload, conferido no cabeçalho antes de alocar memória.
`OutboundQueueCapacity`	`8192`	Capacidade do `Channel` de saída de cada conexão (backpressure).
`RetainedMessagesPerTopic`	`100000`	Janela de mensagens em memória e no journal, por tópico.
`DefaultPrefetch`	`64`	Entregas sem ACK por membro quando o cliente não informa.
`AckTimeout`	`00:00:30`	Prazo para o ACK. Depois dele a mensagem é reentregue.
`MaxDeliveryAttempts`	`5`	Tentativas antes de mandar para `<tópico>.dlq`.
`DeadLetterSuffix`	`.dlq`	Sufixo do tópico de mensagens mortas.
`PersistMessages`	`true`	Liga o journal durável e a gravação dos offsets dos grupos.

EventStore e ReadModel

Chave	Padrão	Descrição
`EventStore:Provider`	`Sqlite`	`Sqlite` ou `Postgres`.
`EventStore:ConnectionString`	`Data Source=data/brado-events.db`	Conexão do lado de escrita.
`EventStore:SnapshotEvery`	`50`	Grava um snapshot a cada N eventos de um stream. Zero desliga.
`ReadModel:Provider`	`Sqlite`	Banco do lado de leitura. Pode ser diferente do de escrita.
`ReadModel:ConnectionString`	`Data Source=data/brado-read.db`	Conexão do modelo de leitura (EF Core).

Copyright (©) 2026 Clara Portes

Messaging, Outbox, Projections, Saga e Simulation

Chave	Padrão	Descrição
`Messaging:Mode`	`InProcess`	`InProcess` usa o broker do próprio processo; `Tcp` conecta em `BrokerHost:BrokerPort` (padrão `localhost:7420`).
`Outbox:BatchSize`	`200`	Mensagens reservadas por ciclo.
`Outbox:PollingInterval`	`00:00:00.500`	Espera máxima entre ciclos. Um append acorda o worker antes disso.
`Outbox:Lease`	`00:00:30`	Por quanto tempo um worker reserva um lote.
`Outbox:BaseRetryDelay / MaxRetryDelay`	`1 s / 5 min`	Backoff exponencial com jitter de ±20%.
`Outbox:DispatchedRetention`	`7 dias`	Mensagens entregues mais antigas que isso são removidas.
`Projections:BatchSize`	`500`	Eventos projetados por transação. `PollingInterval` padrão: 1 s.
`Saga:Enabled / Topic`	`true / orders.events`	Liga a saga e define o tópico que ela consome.
`Saga:PaymentFailureRate / StockFailureRate`	`0.08 / 0.10`	Taxas de falha dos serviços simulados. `SimulatedLatencyMs` padrão: 120.
`Simulation:Ambient`	`false (true em Development)`	Tráfego contínuo para demonstrações: `AmbientOrdersPerMinute` 12 e `AmbientMessagesPerSecond` 40.

Exemplo de sobrescrita por variáveis de ambiente:

```
EventStore__Provider=Postgres
EventStore__ConnectionString="Host=db;Database=brado;Username=brado;Password=***"
Broker__MaxDeliveryAttempts=3
```

4 Conceitos do broker

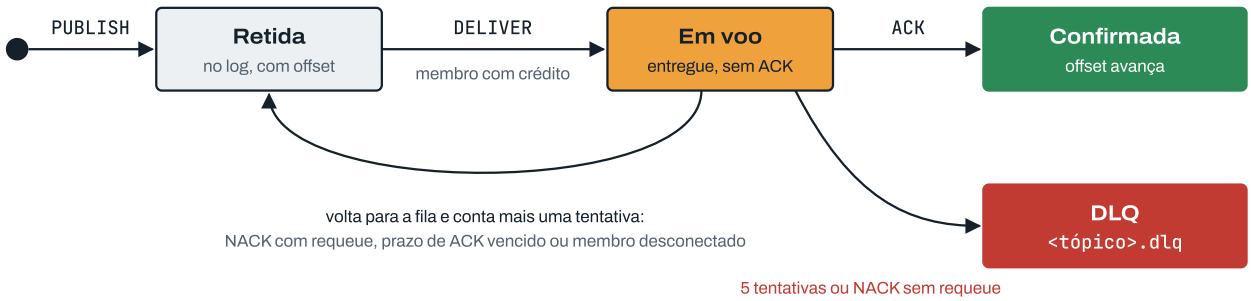
Cada tópico é um log append-only. Sobre esse log, os grupos de consumo dividem as mensagens como numa fila.

Conceito	Como funciona no Brado
Tópico	Log com offsets crescentes a partir de 0, criado no primeiro uso. O nome aceita letras, dígitos, `.`, `-` e `_`, com até 200 caracteres.
Retenção	Cada tópico guarda as últimas `RetainedMessagesPerTopic` mensagens. O descarte é feito em blocos de 10%, para manter o custo constante.

Copyright (©) 2026 Clara Portes

Conceito	Como funciona no Brado
Grupo de consumo	Conjunto de membros que dividem as mensagens. Grupos diferentes recebem todas as mensagens do tópico, cada um no seu ritmo.
Membro	Uma assinatura TCP ou local. As entregas seguem round-robin entre os membros que têm crédito.
Prefetch	Créditos por membro: quantas mensagens podem estar entregues e ainda sem ACK.
ACK	Confirma o offset e devolve o crédito. O offset confirmado do grupo é o menor offset ainda pendente (confirmação contígua).
NACK	Com `Requeue`, a mensagem volta para a fila. Sem ele, ou depois do limite de tentativas, vai para a DLQ.
Timeout	Entregas sem ACK depois de `AckTimeout` voltam para a fila. A manutenção que confere isso roda a cada segundo.
Posição inicial	`Latest` (−1) começa no próximo offset; `Earliest` (0) relê tudo o que ainda está retido. Só vale para grupos novos.
Persistência	Com `PersistMessages`, as mensagens vão para `broker_messages` em lotes de até 512 e os offsets dos grupos para `consumer_offsets` a cada 5 s.

Ciclo de uma mensagem



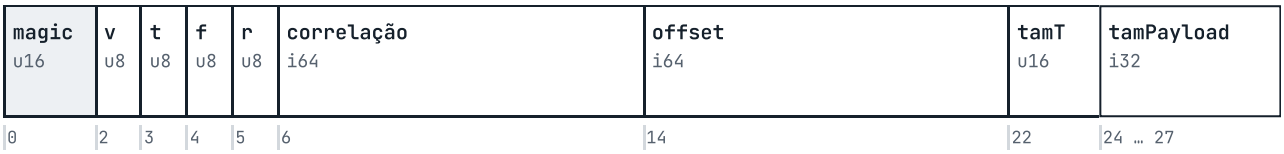
Âmbar marca a mensagem em voo, verde a confirmada e vermelho a que foi para a fila de erros. A flag `Redelivered` acompanha toda reentrega.

Copyright (©) 2026 Clara Portes

5 Protocolo TCP

Quadros binários big-endian com cabeçalho fixo de 28 bytes. O cabeçalho é validado antes de qualquer alocação e o corpo pode ser lido sem cópia.

Cabeçalho



Depois do cabeçalho vêm o tópico (`tamTópico` bytes em UTF-8) e o payload (`tamPayload` bytes).

Bytes	Campo	Tipo	Descrição
0–1	magic	u16	0xA7E5. Qualquer outro valor encerra a conexão.
2	versão	u8	1.
3	tipo	u8	Tipo do quadro (tabela abaixo).
4	flags	u8	Requeue = 1 (NACK), Redelivered = 2 (DELIVER).
5	reservado	u8	Zero.
6–13	correlação	i64	Casa requisição e resposta. Em DELIVER, ACK e NACK é o id da assinatura.
14–21	offset	i64	Offset atribuído (PUBLISH_ACK) ou entregue (DELIVER, ACK, NACK).
22–23	tamTópico	u16	Bytes UTF-8 do tópico, até 512.
24–27	tamPayload	i32	Bytes do payload, até MaxPayloadBytes.

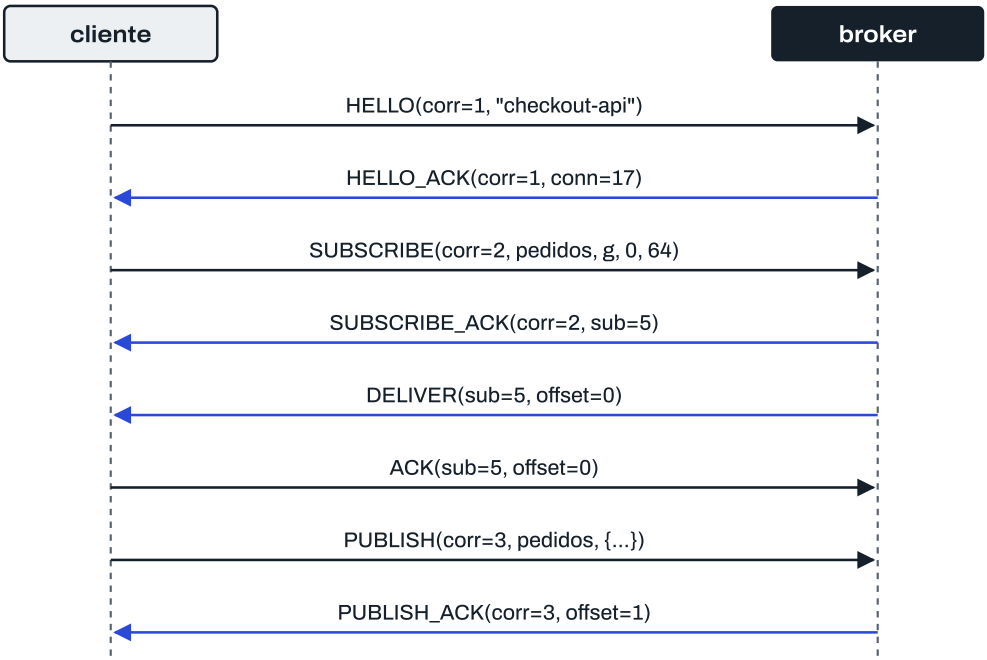
Tipos de quadro

Código	Tipo	Direção	Corpo
0x01 / 0x02	HELLO / HELLO_ACK	cliente ⇄ broker	Nome do cliente (UTF-8) / id da conexão (i64).
0x10 / 0x11	PUBLISH / PUBLISH_ACK	cliente ⇄ broker	Payload livre / offset no cabeçalho.
0x20 / 0x21	SUBSCRIBE / SUBSCRIBE_ACK	cliente ⇄ broker	| tamGrupo u16 | grupo | início i64 | prefetch i32 | / id da assinatura (i64).
0x22	UNSUBSCRIBE	cliente → broker	Correlação = id da assinatura.
0x30	DELIVER	broker → cliente	Payload; correlação = assinatura; offset no cabeçalho.
0x31 / 0x32	ACK / NACK	cliente → broker	Correlação = assinatura; offset; flag Requeue no NACK.
0x40 / 0x41	PING / PONG	os dois lados	Vazio. Usado como keep-alive.

Copyright (©) 2026 Clara Portes

Código	Tipo	Direção	Corpo
`0x7F`	`ERROR`	broker → cliente	Mensagem UTF-8 com o erro de protocolo ou de validação.

Sequência típica



Em preto, quadros do cliente; em cobalto, respostas e entregas do broker.

> **Ordem garantida.** O `SUBSCRIBE_ACK` entra na fila de saída antes da primeira entrega, e o cliente registra a assinatura assim que lê o ACK. Por isso nenhuma entrega chega para uma assinatura que o cliente ainda não conhece.

Copyright (©) 2026 Clara Portes

6 Cliente .NET

`BradoConnection` cuida da conexão, da correlação de respostas, do keep-alive e do consumo com ACK automático.

Conectar e publicar

```
await using var connection = await BradoConnection.ConnectAsync(new BradoClientOptions
{
    Host = "localhost",
    Port = 7420,
    ClientName = "checkout-api",
    RequestTimeout = TimeSpan.FromSeconds(10)
});

long offset = await connection.PublishAsync("pedidos.criados", "texto simples");
offset = await connection.PublishJsonAsync("pedidos.criados", new { pedido = 42, total =
359.30m });
```

Também existe a forma curta, com host, porta e nome do cliente: `BradoConnection.ConnectAsync("localhost", 7420, "checkout-api")`.

Consumir

```
await using var subscription = await connection.SubscribeAsync(
    topic: "pedidos.criados",
    group: "faturamento",
    handler: async (message, ct) ⇒
    {
        var dados = message.AsJson<PedidoCriado>();
        await faturamento.EmitirAsync(dados, ct);
    },
    prefetch: 64,
    startOffset: StartPosition.Earliest);
```

- O handler roda em sequência dentro de cada assinatura. Para processar em paralelo, abra mais assinaturas no mesmo grupo, numa conexão ou em várias.
- Se o handler termina normalmente, o cliente envia ACK. Se lança exceção, envia NACK com requeue. `message.Redelivered` avisa quando é uma reentrega.
- A entrega é at-least-once, então o handler precisa ser idempotente.
- `PingAsync()` mede a latência. O keep-alive roda sozinho a cada 15 s.

Gerar carga pelo protocolo

O projeto `samples/Brado.Demo` usa o mesmo cliente para gerar tráfego:

Copyright (©) 2026 Clara Portes

```
dotnet run --project samples/Brado.Demo -- run
dotnet run --project samples/Brado.Demo -- publish --topic demo.metricas --rate 2000 --publishers 4
dotnet run --project samples/Brado.Demo -- subscribe --topic demo.metricas --group painel --consumers 3
dotnet run --project samples/Brado.Demo -- bench --messages 200000 --size 256
```

Cada `subscribe` com o mesmo `--group` é mais um membro dividindo as mensagens. Grupos diferentes recebem tudo.

7 API HTTP

Minimal APIs que expõem o lado de escrita (comandos), o de leitura (consultas), o event store e o broker.

Método	Rota	Descrição
POST	`/api/orders`	Cria um pedido. Corpo: `customerRef`, `items[]`, `currency`, `submit`. Com `submit: true` a saga começa logo.
POST	`/api/orders/{id}/items`	Adiciona item (`sku`, `quantity`, `unitPrice`).
POST	`/api/orders/{id}/submit`	Envia o pedido e inicia a saga.
POST	`/api/orders/{id}/cancel`	Cancela. Se o pedido já foi pago, exige o estorno antes.
GET	`/api/orders`	Lista paginada do modelo de leitura (`status`, `page`, `pageSize`).
GET	`/api/orders/{id}`	Detalhe do pedido com os itens.
GET	`/api/sales/daily`	Vendas e estornos por dia (`days`).
GET	`/api/read-model`	Totais por status, receita e checkpoint.
POST	`/api/projections/rebuild`	Apaga o modelo de leitura e reconstrói por replay.
GET	`/api/streams`	Streams recentes (`search`, `skip`, `take`).
GET	`/api/streams/{id}/events`	Histórico completo de um stream.
GET	`/api/streams/{id}/replay`	Eventos do stream com o estado depois de cada versão (`toVersion` opcional).
GET	`/api/events`	Log global a partir de `after` (`take` opcional).
GET	`/api/events/stats`	Contagem de eventos por tipo.
GET	`/api/broker`	Fotografia do broker: métricas, tópicos, grupos e conexões.
GET	`/api/broker/topics/{t}/messages`	Mensagens de um tópico (`from`, `take`; padrão 50), com os primeiros 400 bytes do payload.
POST	`/api/broker/topics/{t}/publish`	Publica o corpo bruto da requisição e devolve o offset.

Copyright (©) 2026 Clara Portes

Método	Rota	Descrição
GET	`/api/outbox`	Estado do outbox: pendentes, entregues, falhas, lotes e último erro.
GET	`/api/sagas`	Sagas recentes (`take`; padrão 20).
GET	`/api/overview`	A mesma fotografia que o painel recebe, numa única resposta.
GET	`/api/stream`	Server-Sent Events: um evento `snapshot` por segundo.
POST	`/api/simulate`	Gera pedidos e mensagens (`orders`, padrão 25, até 2.000; `messages`, padrão 600, até 200.000).
GET	`/health`	Verificação de vida.

Erros

Toda falha volta como JSON com `status`, `title` e `detail` (o detalhe é omitido no erro 500).

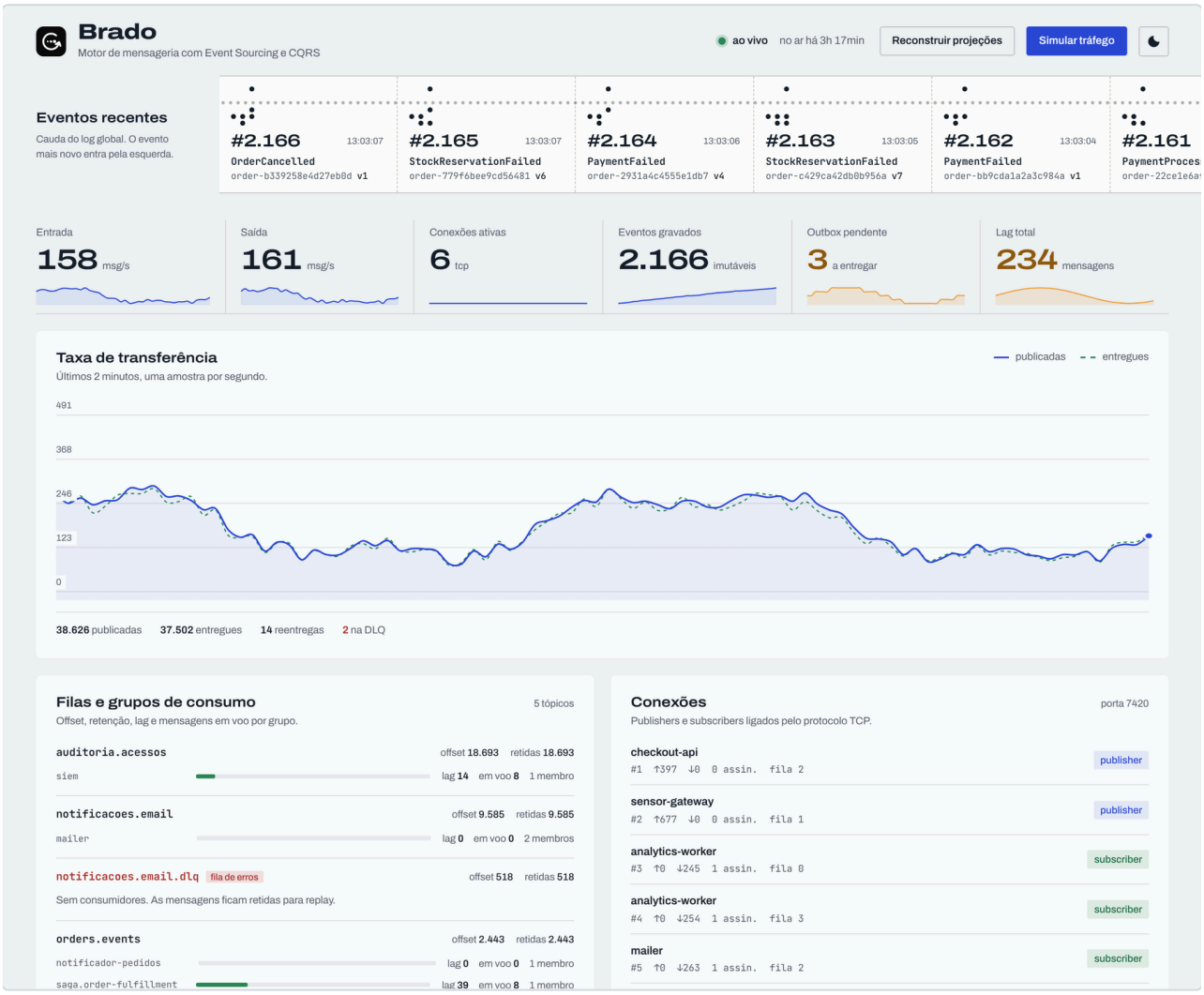
Situação	Título	HTTP
Regra de negócio violada (`DomainException`)	Regra de negócio violada	**422**
Pedido inexistente	Pedido não encontrado	**404**
Conflito de versão depois de 3 tentativas	Conflito de concorrência	**409**
Entrada ou nome de tópico inválido	Requisição inválida	**400**
Qualquer outra exceção	Erro interno	**500**

```
curl -X POST http://localhost:5080/api/orders -H "Content-Type: application/json" \
  -d '{"customerRef":"CLI-1001","submit":true,"items":[{"sku":"LIV-ARQ-
001","quantity":2,"unitPrice":129.90}]}'
```

Copyright (©) 2026 Clara Portes

8 Painel de monitoramento

Interface em HTML, CSS e JavaScript puros, servida pelo próprio servidor e alimentada por Server-Sent Events. A aba do navegador se chama “Painel do Brado”.



Painel no tema claro, com tráfego de demonstração.

Área	O que mostra
Eventos recentes	Faixa em largura total logo abaixo do topo, desenhada como uma fita: a cauda do log global, com o evento mais novo entrando pela esquerda. Cada quadro traz o número de sequência, o tipo do evento, o stream e a hora.
Indicadores	Entrada e saída em msg/s, conexões ativas, eventos gravados, outbox pendente e lag total, cada um com mini-gráfico dos últimos 60 s.
Taxa de transferência	Mensagens publicadas e entregues nos últimos 2 minutos, uma amostra por segundo, com rolagem contínua e escala que se ajusta suavemente. Junto do gráfico ficam os totais de publicadas, entregues, reentregas e mensagens na DLQ.

Copyright (©) 2026 Clara Portes

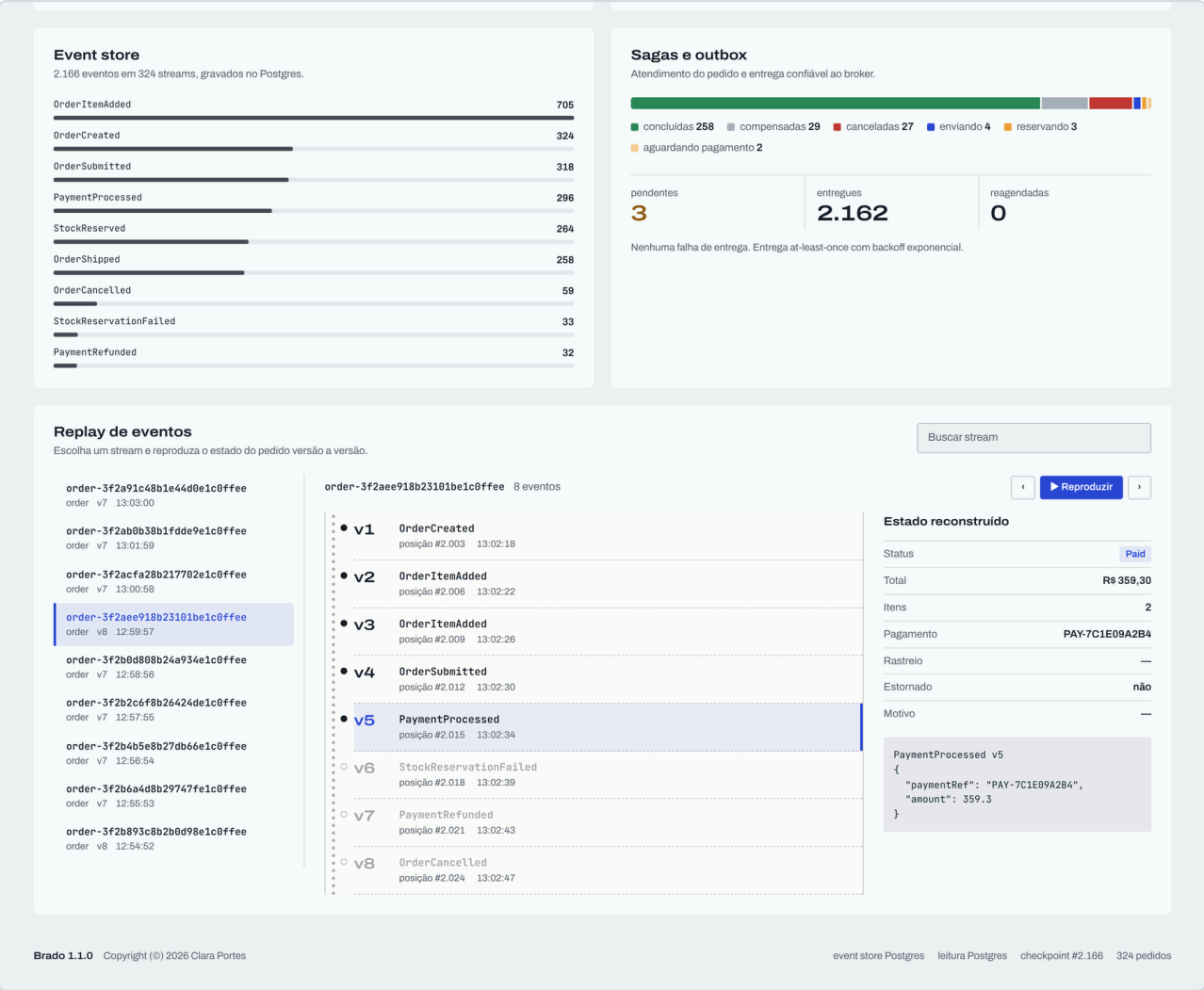
Área	O que mostra
Filas e grupos de consumo	Cada tópico com offset e retenção; cada grupo com lag, mensagens em voo e membros. Tópicos `.dlq` levam a etiqueta “fila de erros”.
Conexões	Clientes TCP com o papel de cada um (publisher, subscriber ou os dois), contadores e fila de saída.
Event store	Banco em uso, eventos por tipo e total de streams.
Sagas e outbox	Distribuição dos estados da saga e situação do outbox: pendentes, entregues, reagendadas e última falha.
Replay de eventos	Busque e escolha um stream e reproduza o estado do pedido versão a versão, com Reproduzir, avançar e voltar. A linha do tempo usa a mesma fita, na vertical, ao lado do estado reconstruído.
Rodapé	Versão em uso (Brado 1.1.0), crédito e informações do servidor.

Botões do topo

- **Simular tráfego** gera 30 pedidos, que percorrem a saga, e 1.500 mensagens de telemetria com consumidores de exemplo.
- **Reconstruir projeções** apaga o modelo de leitura e o refaz a partir do log.
- O botão de tema alterna entre claro e escuro. Sem escolha manual, o painel segue o sistema; a escolha fica guardada no navegador.

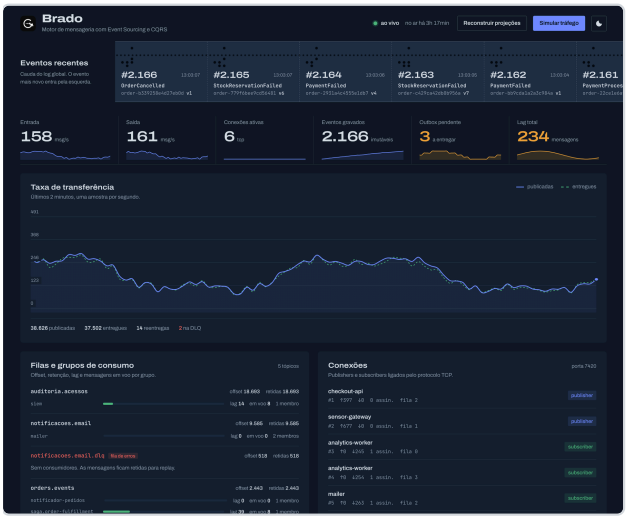
Copyright (©) 2026 Clara Portes

Replay de eventos

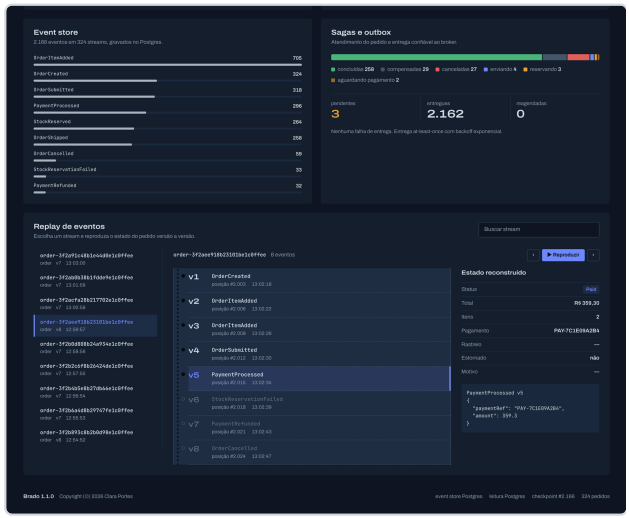


Replay de um pedido: a lista de streams, a fita vertical com os eventos do stream e o estado reconstruído na versão escolhida.

Tema escuro



Painel.



Replay.

9 Event Sourcing e CQRS

O estado do pedido nunca é gravado. Ele é calculado a partir da sequência de eventos do stream, que não muda depois de gravada.

Agregado Order

Comando	Exige	Evento
`CreateOrder`	Cliente e moeda válidos	`OrderCreated`
`AddOrderItem`	Status `Draft`	`OrderItemAdded`
`SubmitOrder`	`Draft` com itens	`OrderSubmitted`
`RecordPayment`	`Submitted` e valor igual ao total	`PaymentProcessed`
`RecordPaymentFailure`	`Submitted`	`PaymentFailed`
`ReserveStock`	`Paid`	`StockReserved`
`RecordStockFailure`	`Paid`	`StockReservationFailed`
`RefundPayment`	Pago e ainda não estornado	`PaymentRefunded`
`ShipOrder`	`StockReserved`	`OrderShipped`
`CancelOrder`	Não enviado; estornado, se foi pago	`OrderCancelled`

Escrita

1. O `CommandDispatcher` resolve o handler por injeção de dependência e repete até 3 vezes quando recebe `ConcurrencyException`.
2. O repositório carrega o snapshot mais recente e os eventos gravados depois dele.
3. O agregado valida o comando e gera os eventos. O repositório grava eventos e outbox numa transação, com a versão esperada.
4. A cada `SnapshotEvery` eventos, um snapshot novo é salvo.

Leitura

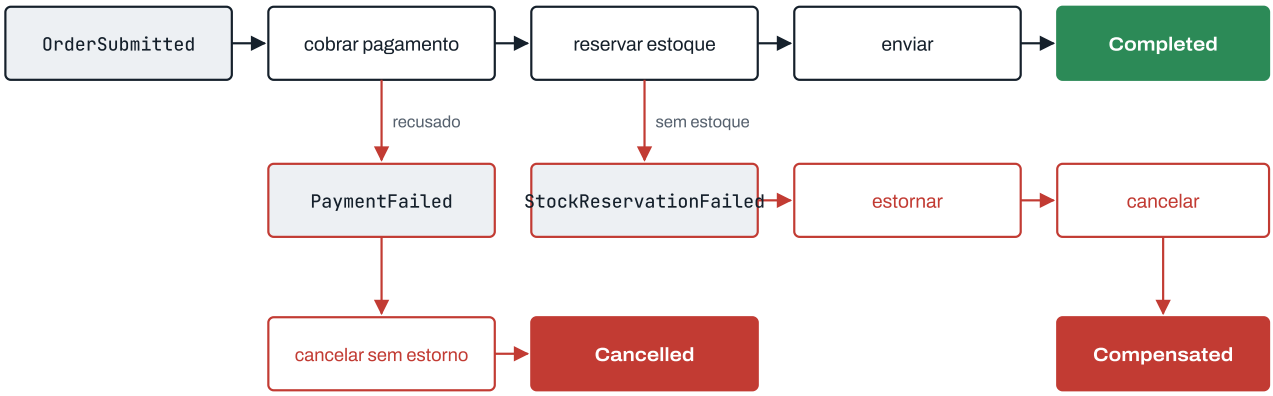
O `ProjectionWorker` lê o log global a partir do checkpoint, aplica as projeções (`OrderSummaryProjection` e `DailySalesProjection`) e grava as visões e o checkpoint na mesma transação. Assim cada evento é projetado exatamente uma vez. O worker acorda logo depois de cada append feito no processo e, sem sinal, uma vez por segundo.

> **Reconstrução.** O modelo de leitura é derivado do log, então pode ser apagado a qualquer momento: `POST /api/projections/rebuild` zera as visões e o replay recria tudo.

Copyright (©) 2026 Clara Portes

10 Saga de atendimento

Uma saga orquestrada coordena pagamento, estoque e envio sem transação distribuída. Quando um passo falha, ela compensa os anteriores.



Linha de cima: caminho feliz. Em vermelho: falhas e compensação. Pagamento recusado cancela sem estorno; falta de estoque estorna e depois cancela.

Evento recebido	Estado da saga	Ação
`OrderSubmitted`	AwaitingPayment	Cobra no gateway e envia `RecordPayment` ou `RecordPaymentFailure`.
`PaymentProcessed`	ReservingStock	Reserva no estoque e envia `ReserveStock` ou `RecordStockFailure`.
`StockReserved`	Shipping	Gera o código de rastreio e envia `ShipOrder`.
`OrderShipped`	Completed	Fim.
`PaymentFailed`	Cancelled	Envia `CancelOrder`.
`StockReservationFailed`	Compensating	Estorna no gateway e envia `RefundPayment`.
`PaymentRefunded`	Compensating	Envia `CancelOrder`.
`OrderCancelled`	Compensated ou Cancelled	Fim.

A saga consome `orders.events` no grupo `saga.order-fulfillment`. Antes de tratar uma mensagem, ela consulta `processed_messages`; depois de tratar, registra o id ali. Comandos repetidos esbarram nas invariantes do agregado, e os serviços simulados são idempotentes por pedido.

Copyright (©) 2026 Clara Portes

11 Outbox e garantias de entrega

Evento e mensagem nascem na mesma transação. A entrega ao broker acontece depois, com novas tentativas e sem perda.

1. **Gravação atômica.** `SqlEventStore.AppendAsync` insere os eventos e as linhas do outbox na mesma transação.
2. **Reserva com lease.** O dispatcher marca até `BatchSize` mensagens com `locked_until`. No PostgreSQL usa `FOR UPDATE SKIP LOCKED`, o que permite vários servidores trabalhando ao mesmo tempo.
3. **Publicação em ordem.** As mensagens do lote saem na ordem do `id`.
4. **Confirmação.** Cada sucesso recebe `dispatched_at`. Cada falha incrementa `attempts` e agenda `next_attempt_at` com backoff exponencial e jitter.
5. **Limpeza.** A cada hora, mensagens entregues há mais de `DispatchedRetention` são removidas.

Garantia	Como é obtida
Sem perda	A mensagem só sai do outbox depois de publicada. Uma falha de rede apenas reagenda.
At-least-once	Se a queda acontece entre publicar e confirmar, a mensagem é reenviada. Os consumidores deduplicam pelo `messageId`.
Sem dual-write	Não existe um momento em que o evento foi gravado e a mensagem não.
Ordem por stream	Os lotes saem em sequência pelo `id`, e os consumidores podem conferir `streamVersion`.

Garantias em cada camada

Camada	Mecanismo	Comportamento padrão
Publicação	`PUBLISH_ACK` com offset	O publisher só considera enviado depois que o broker grava no log.
Prefetch	Créditos por membro	Até 64 mensagens sem ACK por membro.
Prazo de ACK	Manutenção a cada segundo	Mensagem sem ACK em 30 s volta para a fila.
Novas tentativas	Contador por offset	Até 5 entregas, marcadas com a flag `Redelivered`.
Fila de erros	Tópico `<tópico>.dlq`	Recebe a mensagem quando as tentativas acabam.
Outbox	Backoff exponencial com jitter	Começa em 1 s, dobra a cada falha e para em 5 min, com 20% de variação.
Persistência	Journal em lote e `consumer_offsets`	Mensagens em lotes de até 512; offsets dos grupos a cada 5 s.

Copyright (©) 2026 Clara Portes

12 Testes e benchmarks

Testes com xUnit v3, integração com um PostgreSQL real via Testcontainers e medições com BenchmarkDotNet.

```
dotnet test tests/Brado.UnitTests
dotnet test tests/Brado.IntegrationTests
dotnet run -c Release --project benchmarks/Brado.Benchmarks -- --filter "*"
```

Suíte	Casos principais
`ProtocolTests`	Ida e volta do codec, tamanho, magic inválido, payload acima do limite, leitura sequencial, SUBSCRIBE e nomes de tópico.
`OrderTests`	Caminho feliz, invariantes, compensação, reidratação e snapshot.
`BrokerTests`	Offsets, competição entre membros, grupos independentes, prefetch, commit contíguo, NACK, DLQ, troca de membro, timeout e posição inicial.
`SqliteEventStoreTests`	Append e leitura, concorrência, imutabilidade, outbox atômico e reagendado, repositório com snapshot.
`CqrsAndSagaTests`	Saga com sucesso, compensação e falha de pagamento; projeção e rebuild; backoff.
`BrokerTcpTests`	Pub/sub real em TCP, offsets, divisão de carga, reentrega, desconexão e erro de protocolo.
`PostgresTests`	Ordem global, 8 escritores concorrentes, trigger, `SKIP LOCKED` com 4 workers, outbox sem perda e journal.

Benchmark	Mede
`FrameCodecBenchmarks`	Codificação em `Span`, leitura de cabeçalho e decodificação, de 64 B a 16 KiB.
`BrokerBenchmarks`	Publicação sem consumidores e publicação com consumo local.
`ChannelBenchmarks`	`Channel` limitado contra ilimitado, com leitor único.
`EventStoreBenchmarks`	Append com outbox no SQLite, com 1 e com 10 eventos.

O cliente de demonstração também mede a vazão de ponta a ponta: `dotnet run --project samples/Brado.Demo -- bench --messages 200000 --size 256`.

A integração contínua (`.github/workflows/ci.yml`) compila tudo e roda as duas suítes a cada envio.

Copyright (©) 2026 Clara Portes

13 Operação e solução de problemas

Recomendações para produção e o que fazer diante dos sintomas mais comuns.

Produção

- Use PostgreSQL na escrita e, de preferência, um banco separado para a leitura.
- Deixe o broker em rede interna. O protocolo não tem autenticação; para expor, ponha um proxy TCP com TLS na frente da porta 7420.
- Para vários servidores, use `Messaging:Mode=Tcp` apontando para um broker central. O outbox com `SKIP LOCKED` divide a entrega sem duplicar reservas.
- Faça backup do event store com `pg_dump` ou PITR. O modelo de leitura não precisa de backup, porque pode ser reconstruído.
- Monitore `/health`, o lag dos grupos e o outbox pendente em `/api/broker` e `/api/outbox`.
- As credenciais do `docker-compose.yml` e do `appsettings.Postgres.json` são de desenvolvimento local. Troque-as antes de expor o banco.

Sintomas comuns

Sintoma	Causa provável	O que fazer
Lag crescendo em um grupo	Consumidores lentos ou prefetch baixo	Aumente os membros do grupo ou o prefetch e verifique erros no handler.
Mensagens indo para a DLQ	Handler falhando repetidamente	Inspecione `<tópico>.dlq` em `/api/broker/topics/.../messages` e corrija o consumidor.
Outbox pendente que não diminui	Broker inacessível no modo `Tcp`	Veja `lastError` em `/api/outbox`. As mensagens saem quando o broker voltar.
HTTP 409 em comandos	Muitas escritas no mesmo pedido	É esperado sob disputa. O cliente pode repetir a requisição.
Modelo de leitura desatualizado	Projeção com erro	Verifique os logs do `ProjectionWorker` e use `/api/projections/rebuild`.
`database is locked` no SQLite	Muita escrita concorrente	Migre para PostgreSQL. O SQLite já usa WAL e `BEGIN IMMEDIATE`.

Copyright (©) 2026 Clara Portes

14 Automação de commits

Um workflow do GitHub Actions registra um checkpoint por execução, cinco vezes ao dia.

Execução	Horário de Brasília	Cron (UTC)
1	09:17	`17 12 * * *`
2	11:43	`43 14 * * *`
3	14:08	`8 17 * * *`
4	16:52	`52 19 * * *`
5	20:26	`26 23 * * *`

- Arquivo: `.github/workflows/daily-activity.yml`. Cada execução acrescenta uma linha em `.github/activity/AAAA-MM.md` e faz um commit.
- O autor do commit é o dono do repositório, com o e-mail `noreply` do GitHub montado em tempo de execução. Nenhum dado pessoal fica no código.
- A concorrência é controlada (`concurrency`) e o push usa `pull --rebase` com até 3 tentativas.
- Commits feitos com `GITHUB_TOKEN` não disparam o CI, e o CI também ignora `.github/activity/**`.
- Para rodar na hora: aba Actions, Registro diário, Run workflow. Para pausar: Disable workflow.
- O GitHub pode atrasar execuções agendadas nos horários de pico. Os minutos fora da hora cheia reduzem esse efeito.

15 Novidades da 1.1.0

A versão 1.1.0 muda a aparência do painel e dos manuais. O broker, o protocolo, a API e o banco de dados continuam iguais aos da 1.0.0.

- Painel redesenhado a partir da ideia de uma fita de teletipo. A área de eventos recentes virou uma fita contínua logo abaixo do topo, com o número de sequência de cada evento e furos que representam esse número em binário. A mesma fita aparece na vertical na linha do tempo do replay.
- Painéis lisos, sem sombra e sem borda; a hierarquia vem do tamanho, do peso e do espaço. Os cantos variam conforme o elemento: 6 px nos painéis, 3 px em botões e campos.
- Cores novas, com significado fixo: cobalto para ações e mensagens publicadas, âmbar para o que está em voo ou pendente, verde para o que foi confirmado e vermelho para falhas, DLQ e cancelamentos. O tema escuro usa as mesmas funções com tons próprios.

Copyright (©) 2026 Clara Portes

- Tipografia nova: Archivo, com títulos em largura expandida, e JetBrains Mono só para código (tópicos, ids de stream, tipos de evento, offsets e JSON). As duas fontes passaram a ser servidas pelo próprio Brado, em `wwwroot/fonts`, sem depender do Google Fonts.
- Capturas de tela refeitas com o painel novo, nos temas claro e escuro.
- Manuais reescritos e com novo projeto gráfico. A API ganhou a documentação de rotas que já existiam e não apareciam aqui: `/api/streams/{id}/events`, `/api/events/stats`, `/api/broker/topics/{t}/messages`, `/api/outbox`, `/api/sagas` e `/api/overview`.

Paleta do tema claro

Névoa Fundo #EDF0F3	**Tinta** Texto e furos #15202B	**Cobalto** Ação #2A49D6
Âmbar Em voo #EFA23B	**Verde-sinal** Confirmado #2C8A57	**Vermelho-sinal** Falha #C23B33

No tema escuro o fundo é `#0F1824`, o texto `#D7DEE6` e o cobalto fica mais claro, `#6F8BFF`. A logo não mudou.

Copyright (©) 2026 Clara Portes